PHY F376: A study of hybrid classical-quantum algorithms

Week 2 (27/8/2026) - A Code-Level Study of the SQD Workflow: N2 Tutorial and Chemistry Function Template

Khushi Pandey (2023B5A70951G)

Overview of This Week's Work

Following last week's conceptual overview of sample-based quantum diagonalization (SQD), this week I went through the accompanying Jupyter notebooks line by line: the N2 tutorial, which runs the SQD workflow directly, and the chemistry function-template guide, which packages the same workflow as a deployable service. The goal was to move from understanding what SQD does to understanding how each part of it is actually implemented in code.

Building the Quantum Circuit

The first half of the workflow is really classical quantum chemistry dressed up as circuit input. PySCF is used to define the molecule, select an active space, and run a Hartree-Fock calculation, from which the one- and two-electron integrals are extracted. A classical CCSD calculation is then run purely to obtain t_1 and t_2 amplitudes, which are not used directly as an energy but as an initial guess for the quantum circuit's parameters. I found this a useful realization: SQD does not start from a "blank" ansatz, but from one already informed by a cheaper classical calculation. The LUCJ circuit is then built with `ffsim` on top of a Hartree-Fock reference state, using the Jordan-Wigner mapping to represent each spin-orbital as a qubit, so a measured bitstring directly corresponds to an electron configuration. Because LUCJ is designed to respect the hardware's own qubit connectivity, the pass manager and coupling map (heavy-hex, in IBM's case) have to be built before the ansatz itself, which was a somewhat unusual order of operations to me at first.

Understanding the SQD Post-Processing Step

Once the circuit is transpiled and sampled, the resulting bitstrings are handed to the diagonalize_fermionic_hamiltonian function. Going through this call argument by argument helped clarify the mechanics of self-consistent configuration recovery: bitstrings that do not conserve the correct particle number are corrected using the current best guess of orbital occupancies, valid configurations are grouped into randomly drawn batches, each batch's Hamiltonian is diagonalized classically using a Davidson solver, and the lowest-energy batch updates the occupancy guess for the next round. Seeing this expressed as actual function arguments (`samples_per_batch`, `num_batches`, `max_iterations`, and so on) made the earlier, more abstract description of the algorithm concrete for me.

From Notebook to Deployable Workflow

The chemistry function-template guide showed how this same pipeline can be wrapped into a reusable Qiskit Function and run remotely through Qiskit Serverless, rather than executed cell by cell. Last week I described these options in the abstract; this week I traced exactly how each one maps to a specific line of the pipeline. Working through the `molecule`, `solvent_options`, `lucj_options`, and `sqd_options` dictionaries used for the methanol example clarified how each user-facing setting maps onto a step of the workflow: the active space and AVAS orbital selection feed into Step 1, the qubit layout and shot count into Step 2, and the batching and iteration parameters into the final SQD diagonalization step. This also introduced the IEF-PCM implicit solvent model, which adds a solvation free energy term to account for the surrounding solvent without simulating it explicitly.

Concluding Remarks

This week's focus on the code itself, rather than only the underlying theory, was necessary to see how the pieces actually fit together: the molecular Hamiltonian, the ansatz construction, the sampling step, and the classical diagonalization all connect through specific, well-defined function calls rather than being separate ideas. With this understanding in place, I feel ready to move on to working with an actual Hamiltonian for the project.